STREAM PRACTICE: ANSWER

09 September 2025 12:30

✅ Easy Level (1–10)

```
import java.util.*;
import java.util.stream.*;
public class EasyLevel {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1,2,3,4,5,10,12,15,20);
        List<String> words = Arrays.asList("apple","banana","cat","elephant","dog");
        List<String> names = Arrays.asList("Alex","Bob","Annie","Charlie","Aman");
// 1. Even numbers
        numbers.stream().filter(n -> n % 2 == 0).forEach(System.out::println);
// 2. Uppercase
        words.stream().map(String::toUpperCase).forEach(System.out::println);
// 3. Count > 10
        long count = numbers.stream().filter(n -> n > 10).count();
        System.out.println(count);
// 4. Names starting with A
        names.stream().filter(n -> n.startsWith("A")).forEach(System.out::println);
// 5. Square each number
        numbers.stream().map(n -> n * n).forEach(System.out::println);
// 6. Sort ascending
        numbers.stream().sorted().forEach(System.out::println);
// 7. First element
        numbers.stream().findFirst().ifPresent(System.out::println);
// 8. Filter words length > 5
        words.stream().filter(w -> w.length() > 5).forEach(System.out::println);
// 9. Remove duplicates
        numbers.stream().distinct().forEach(System.out::println);
// 10. Sum using reduce
        int sum = numbers.stream().reduce(0, Integer::sum);
        System.out.println(sum);
    }
}
```

✅ Medium Level (11–20)

```
import java.util.*;
        import java.util.stream.*;
class Employee {
    String name;
    double salary;
    Employee(String n, double s) { name=n; salary=s; }
    public String toString() { return name + " : " + salary; }
}
public class MediumLevel {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
                new Employee("John", 40000),
                new Employee("Jane", 60000),
                new Employee("Alex", 75000)
        );
        List<Integer> numbers = Arrays.asList(5, 1, 2, 8, 10, 15, 20);
```

```
        List<String> words = Arrays.asList("dog","elephant","cat","ball");
        List<String> sentences = Arrays.asList("Hello world", "Java streams practice");
// 1. Salary > 50k
        employees.stream().filter(e -> e.salary > 50000).forEach(System.out::println);
// 2. Max number
        numbers.stream().max(Integer::compare).ifPresent(System.out::println);
// 3. Min number
        numbers.stream().min(Integer::compare).ifPresent(System.out::println);
// 4. Words reverse sorted
        words.stream().sorted(Comparator.reverseOrder()).forEach(System.out::println);
// 5. Second largest number

numbers.stream().sorted(Comparator.reverseOrder()).skip(1).findFirst().ifPresent(System.out::println);
// 6. Count words with 'e'
        long count = words.stream().filter(w -> w.contains("e")).count();
        System.out.println(count);
// 7. Average salary
        double avg = employees.stream().mapToDouble(e -> e.salary).average().orElse(0);
        System.out.println(avg);
// 8. Count words per sentence
        sentences.stream().map(s -> s.split(" ").length).forEach(System.out::println);
// 9. Concatenate strings
        String concat = words.stream().reduce("", (a,b) -> a+b);
        System.out.println(concat);
// 10. Group by age
        Map<Integer, List<String>> people = Stream.of(
                new AbstractMap.SimpleEntry<>("John", 25),
                new AbstractMap.SimpleEntry<>("Jane", 30),
                new AbstractMap.SimpleEntry<>("Alex", 25)
        ).collect(Collectors.groupingBy(Map.Entry::getValue,
                Collectors.mapping(Map.Entry::getKey, Collectors.toList())));
        System.out.println(people);
    }
}
```

✅ Hard Level (21–30)

```
import java.util.*;
        import java.util.stream.*;
class Emp {
    int id; String name, dept; double salary;
    Emp(int id, String n, String d, double s) { this.id=id; name=n; dept=d; salary=s; }
    public String toString(){ return name+"("+dept+"):"+salary; }
}
class Transaction {
    int id; double amount; String type;
    Transaction(int i,double a,String t){ id=i; amount=a; type=t; }
}
class Product {
    int id; String name; double price;
    Product(int i,String n,double p){ id=i; name=n; price=p; }
}
class Student {
    String name; int marks;
    Student(String n,int m){ name=n; marks=m; }
}
class Book {
```

```java
    String title, author; int year;
    Book(String t,String a,int y){ title=t; author=a; year=y; }
}
public class HardLevel {
    public static void main(String[] args) {
        List<Emp> emps = Arrays.asList(
                new Emp(1,"John","IT",70000),
                new Emp(2,"Jane","HR",65000),
                new Emp(3,"Alex","IT",90000)
        );
        List<Transaction> tx = Arrays.asList(
                new Transaction(1,100,"Food"),
                new Transaction(2,200,"Travel"),
                new Transaction(3,150,"Food")
        );
        List<List<Integer>> nested = Arrays.asList(
                Arrays.asList(1,2), Arrays.asList(3,4), Arrays.asList(5,6)
        );
        List<Integer> numbers = Arrays.asList(10,20,30,40,50,60,30);
        List<Product> products = Arrays.asList(
                new Product(1,"Laptop",50000),
                new Product(2,"Phone",20000)
        );
        List<Student> students = Arrays.asList(
                new Student("A",90),
                new Student("B",95),
                new Student("C",85)
        );
        List<String> words = Arrays.asList("dog","elephant","cat","ball","hippopotamus");
        List<Book> books = Arrays.asList(
                new Book("Book1","Author1",2020),
                new Book("Book2","Author2",2021),
                new Book("Book3","Author1",2019)
        );
// 1. Highest paid in each dept
        Map<String, Optional<Emp>> highest = emps.stream()
                .collect(Collectors.groupingBy(e -> e.dept,
                        Collectors.maxBy(Comparator.comparingDouble(e -> e.salary))));
        System.out.println(highest);
// 2. Total amount per transaction type
        Map<String, Double> txTotal = tx.stream()
                .collect(Collectors.groupingBy(t -> t.type,
                        Collectors.summingDouble(t -> t.amount)));
        System.out.println(txTotal);
// 3. Flatten list
        List<Integer> flat = nested.stream().flatMap(List::stream).collect(Collectors.toList());
        System.out.println(flat);
// 4. Top 3 highest numbers

numbers.stream().sorted(Comparator.reverseOrder()).distinct().limit(3).forEach(System.out::println);
// 5. Product map name -> price
        Map<String, Double> productMap = products.stream().collect(Collectors.toMap(p -> p.name, p ->
p.price));
        System.out.println(productMap);
// 6. Partition even & odd
        Map<Boolean, List<Integer>> partition = numbers.stream().collect(Collectors.partitioningBy(n ->
n%2==0));
        System.out.println(partition);
```

```
// 7. Student with highest marks
        students.stream().max(Comparator.comparingInt(s -> s.marks)).ifPresent(System.out::println);
// 8. Longest string
        words.stream().max(Comparator.comparingInt(String::length)).ifPresent(System.out::println);
// 9. Group books by author
        Map<String, Long> booksByAuthor = books.stream()
                .collect(Collectors.groupingBy(b -> b.author, Collectors.counting()));
        System.out.println(booksByAuthor);
// 10. Sum of squares of odd numbers
        int sumSquares = numbers.stream().filter(n -> n%2!=0).map(n -> n*n).reduce(0,Integer::sum);
        System.out.println(sumSquares);
    }
}
```